

MultiheadAttention#

<https://pytorch.org/docs/stable/generated/torch.nn.quantizable.MultiheadAttention.html>

Description:

Utility to convert the quantized MHA back to float. The motivation for this is that it is not trivial to convert the weights from the format that is used in the quantized version back to the float. Please, refer to `forward()` for more information.

`query (Tensor)` – map a query and a set of key-value pairs to an output. See “Attention Is All You Need” for more details.

`key (Tensor)` – map a query and a set of key-value pairs to an output. See “Attention Is All You Need” for more details.

Function Signature

```
class torch.nn.quantizable.MultiheadAttention(embed_dim,
num_heads, dropout=0.0, bias=True, add_bias_kv=False,
add_zero_attn=False, kdim=None, vdim=None,
batch_first=False, device=None, dtype=None)[source]#
dequantize()[source]#
forward(query, key, value, key_padding_mask=None,
need_weights=True, attn_mask=None,
average_attn_weights=True, is_causal=False)[source]#
```

Note::

Parameters

Returns:

tuple[torch.Tensor, Optional[torch.Tensor]]

Detailed Documentation

Documentation

Utility to convert the quantized MHA back to float.

The motivation for this is that it is not trivial to convert the weights from the format that is used in the quantized version back to the float.

Please, refer to `forward()` for more information

`query` (Tensor) – map a query and a set of key-value pairs to an output. See “Attention Is All You Need” for more details.

`key` (Tensor) – map a query and a set of key-value pairs to an output. See “Attention Is All You Need” for more details.

`value` (Tensor) – map a query and a set of key-value pairs to an output. See “Attention Is All You Need” for more details.

`key_padding_mask` (Optional[Tensor]) – if provided, specified padding elements in the key will be ignored by the attention. When given a binary mask and a value is True, the corresponding value on the attention layer will be ignored.

`need_weights` (bool) – output `attn_output_weights`.

`attn_mask` (Optional[Tensor]) – 2D or 3D mask that prevents attention to certain positions. A 2D mask will be broadcasted for all the batches while a 3D mask allows to specify a different mask for the entries of each batch.

`tuple[torch.Tensor, Optional[torch.Tensor]]`

`query`: (L,N,E)(L, N, E)(L,N,E) where L is the target sequence length, N is the batch size, E is the embedding dimension. (N,L,E)(N, L, E)(N,L,E) if `batch_first` is True.

`key`: (S,N,E)(S, N, E)(S,N,E), where S is the source sequence length, N is the batch size, E is the embedding dimension. (N,S,E)(N, S, E)(N,S,E) if `batch_first` is True.

`value`: (S,N,E)(S, N, E)(S,N,E) where S is the source sequence length, N is the batch size, E is the embedding dimension. (N,S,E)(N, S, E)(N,S,E) if `batch_first` is True.

`key_padding_mask`: (N,S)(N, S)(N,S) where N is the batch size, S is the source

sequence length. If a BoolTensor is provided, the positions with the value of True will be ignored while the position with the value of False will be unchanged.

attn_mask: 2D mask (L,S)(L, S)(L,S) where L is the target sequence length, S is the source sequence length. 3D mask (N*numheads,L,S)(N*num_heads, L, S)(N*numheads,L,S) where N is the batch size, L is the target sequence length, S is the source sequence length. attn_mask ensure that position i is allowed to attend the unmasked positions. If a BoolTensor is provided, positions with True is not allowed to attend while False values will be unchanged. If a FloatTensor is provided, it will be added to the attention weight.

is_causal: If specified, applies a causal mask as attention mask. Mutually exclusive with providing attn_mask. Default: False.

average_attn_weights: If true, indicates that the returned attn_weights should be averaged across heads. Otherwise, attn_weights are provided separately per head. Note that this flag only has an effect when need_weights=True.. Default: True (i.e. average weights across heads)

attn_output: (L,N,E)(L, N, E)(L,N,E) where L is the target sequence length, N is the batch size, E is the embedding dimension. (N,L,E)(N, L, E)(N,L,E) if batch_first is True.

attn_output_weights: If average_attn_weights=True, returns attention weights averaged across heads of shape (N,L,S)(N, L, S)(N,L,S), where N is the batch size, L is the target sequence length, S is the source sequence length. If average_attn_weights=False, returns attention weights per head of shape (N,numheads,L,S)(N, num_heads, L, S)(N,numheads,L,S).